

consteval blocks

Document #: P3289R1 [[Latest](#)] [[Status](#)]
Date: 2025-01-12
Project: Programming Language C++
Audience: CWG
Reply-to: Wyatt Childers
<wcc@edg.com>
Dan Katz
<dkatz85@bloomberg.net>
Barry Revzin
<barry.revzin@gmail.com>
Daveed Vandevoorde
<daveed@edg.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
- [3 Proposal](#)
- [4 Implementation Status](#)
- [5 Wording](#)
 - [5.1 Feature-Test Macro](#)
- [6 References](#)

§ 1 Revision History

Since [\[P3289R0\]](#), updated wording to make a consteval block distinct from a `static_assert`.

§ 2 Introduction

Several proposals that produce side effects as part of constant evaluation are in flight. That includes [\[P2996R9\] \(Reflection for C++26\)](#) and [\[P2758R3\] \(Emitting messages at compile time\)](#). Such a capability, in turn, quickly gives rise to the desire to evaluate such constant expressions in declarative contexts.

Currently, this effect can be shoe-horned into `static_assert` declarations, but the result looks arcane. For example, P2996 contains the following code in an example:

```
#include <meta>

template<typename... Ts> struct Tuple {
```

```

struct storage;
static_assert(
    is_type(define_class(^storage, {data_member_spec(^Ts)...})));
storage data;

Tuple(): data{} {}
Tuple(Ts const& ...vs): data{ vs... } {}
};

```

Here, `define_class(...)` is a constant expression with a side-effect that we want to evaluate before parsing the member declaration that follows. It works, but it is somewhat misleading: We're not really trying to assert anything; we just want to force that evaluation.

We therefore propose a simple, intuitive syntax to express that we simply want to constant-evaluate a bit of code wherever a `static_assert` declaration could appear by enclosing that code in `consteval { ... }` (a construct we'll call a `consteval` block).

§ 3 Proposal

Formally, we propose that a construct of the form

```

consteval {
    statement-seqopt
}

```

is equivalent to:

```

static_assert(
    (
        []() -> void consteval {
            statement-seqopt
        }(),
        true
    )
);

```

The `static_assert` condition is a *comma-expression* with the first operand an immediately-invoked `consteval` lambda.

Note that this allows a plain `return`; statement or even a `return f();` statement where `f()` has type ``void`. We could go out of our way to disallow that, but we cannot find any benefit in doing so.

With the feature as proposed, the example above becomes:

Status Quo	Proposed
<pre>#include <meta> template<typename... Ts> struct Tuple { struct storage; static_assert(is_type(define_class(^storage, {data_member_spec(^Ts)...}))); storage data; };</pre>	<pre>#include <meta> template<typename... Ts> struct Tuple { struct storage; consteval { define_class(^storage, {data_member_spec(^Ts)...}); } storage data; };</pre>

In this example, there is just a single expression statement being evaluated. However, we are anticipating reflection code where more complex statement sequences will be used (you can see some examples in previous papers, e.g. [\[P1717R0\]__ \(Compile-time Metaprogramming in C++\)](#) [\[P2237R0\] \(Metaprogramming\)](#)).

We did consider other syntax variations such as

- `@eval expression;`
- `@ expression;`
- `@ statement;`

but found those alternatives less general and not worth the slight improvement in brevity.

§ 4 Implementation Status

Implemented in both EDG and Clang.

§ 5 Wording

[Editor's note: The simplest way to do the wording is to add a `consteval` block as a *kind of static_assert-declaration*. That's the minimal diff. However, it's kind of weird to say that a `consteval` block literally is a `static_assert` - plus we need to give specific evaluation guarantees to a `consteval` block ("plainly constant-evaluated"), so we'd rather take a few more words to get somewhere that feels more sensible. Plus this change reduces a lot of duplication between *empty-declaration* and *static_assert-declaration*, which are treated the same in a lot of places anyway.]

Change 6.2 [\[basic.def\]/2](#):

- 2 Each entity declared by a *declaration* is also defined by that *declaration* unless:
- (2.1) — it declares a function without specifying the function's body ([\[dcl.fct.def\]](#)),

- (2.2) — [...]
- (2.13) — it is a ~~static_assert-declaration~~ *vacant-declaration* ([dcl.pre]),
- (2.14) — it is an *attribute-declaration* ([dcl.pre]),
- (2.15) — ~~it is an empty-declaration~~ ([dcl.pre]),
- (2.16) — [...]

Extend the wording for plainly constant-evaluated to allow a `constexpr` block [Drafting note: this is adjusting wording that is added by [P2996R9]]:

- 21pre A non-dependent expression or conversion is *plainly constant-evaluated* if it is not in a complete-class context (11.4.1 [class.mem.general]) and either
- (21.1) — it is the evaluating expression of a *constexpr-block-declaration* (9.1 [dcl.pre]), or
 - (21.2) — it is an initializer of a `constexpr` (9.2.6 [dcl.constexpr]) or `constexpr` (9.2.7 [dcl.constinit]) variable ~~that is not in a complete-class context~~ (11.4.1 [class.mem.general]).

[Note 1: The evaluation of a plainly constant-evaluated expression *E* can produce injected declarations (see below) and happens exactly once (5.2 [lex.phases]). Any such declarations are reachable from a point that follows immediately after *E*. — end note]

Change 9.1 [dcl.pre]:

```

name-declaration:
    block-declaration
    namespace-function-declaration
    function-definition
    friend-type-declaration
    template-declaration
    deduction-guide
    linkage-specification
    namespace-definition
-   empty-declaration
    attribute-declaration
    module-import-declaration

block-declaration:
    simple-declaration
    asm-declaration
    namespace-alias-definition
    using-declaration
    using-enum-declaration
    using-directive
-   static_assert-declaration
    alias-declaration
    opaque-enum-declaration

```

```

+ vacant-declaration

+ vacant-declaration:
+   static_assert-declaration
+   empty-declaration
+   constexpr-block-declaration

static_assert-declaration:
    static_assert ( constant-expression ) ;
    static_assert ( constant-expression , static_assert-message ) ;

+ constexpr-block-declaration:
+   constexpr compound-statement

```

And then after 9.1 [\[dcl.pre\]](#)/13:

13 *Recommended practice*: When a *static_assert-declaration* fails, [...]

* If a *constexpr-block-declaration* is within a template definition, it has no effect. The evaluating expression of a *constexpr-block-declaration* is

```
[]() -> void constexpr compound-statement ()
```

[*Note 2*: This expression is plainly constant-evaluated ([*expr.const*]). — *end note*]

14 An *empty-declaration* has no effect.

Adjust the grammar in 11.4.1 [\[class.mem.general\]](#) and the rule in p3:

```

member-declaration:
    attribute-specifier-seqopt decl-specifier-seqopt member-declarator-
        listopt;
    function-definition
    friend-type-declaration
    using-declaration
    using-enum-declaration
-   static_assert-declaration
+   vacant-declaration
    template-declaration
    explicit-specialization
    deduction-guide
    alias-declaration
    opaque-enum-declaration
-   empty-declaration

```

3 A *member-declaration* does not declare new members of the class if it is

(3.1) — a friend declaration ([*class.friend*]),

(3.2) — a *deduction-guide* ([*temp.deduct.guide*]),

(3.3) — a *template-declaration* whose declaration is one of the above,

- (3.4) — a ~~static_assert-declaration~~;
- (3.5) — a *using-declaration* ([namespace.udecl]) , or
- (3.6) — ~~an empty-declaration~~; a *vacant-declaration*.

And similar in 11.5.2 [\[class.union.anon\]](#)/1. [Editor's note: This refactor allows putting in an *empty-declaration* into an anonymous union, which is kind of a consistency drive by with other classes.]

- 1 [...] Each *member-declaration* in the *member-specification* of an anonymous union shall either define one or more public non-static data members or be a ~~static_assert-declaration~~ *vacant-declaration*. [...]

§ 5.1 Feature-Test Macro

Add to the table in 15.11 [\[cpp.predefined\]](#):

```
__cpp_consteval      202211L
+ __cpp_consteval_block 2025XXL
__cpp_constinit      201907L
```

§ 6 References

[P1717R0] Andrew Sutton, Wyatt Childers. 2019-06-17. Compile-time Metaprogramming in C++. <https://wg21.link/p1717r0>

[P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming. <https://wg21.link/p2237r0>

[P2758R3] Barry Revzin. 2024-05-19. Emitting messages at compile time. <https://wg21.link/p2758r3>

[P2996R9] Wyatt Childers, Peter Dimov, Dan Katz, Barry Revzin, Andrew Sutton, Faisal Vali, and Daveed Vandevoorde. 2025-01-12. Reflection for C++26. <https://wg21.link/p2996r9>

[P3289R0] Daveed Vandevoorde, Wyatt Childers, Barry Revzin. 2024-05-21. Consteval blocks. <https://wg21.link/p3289r0>